

Algoritmo per il taglio minimo

Teorema 22.11 L'algoritmo **Randomized Min-Cut** produce un taglio minimo con probabilità $\geq \frac{2}{n(n-1)}$.

Dim.

Sia C uno dei tagli minimi del grafo originale $G = (V, E)$, e sia k la cardinalità di C .

Per il Lemma 22.10, la probabilità che C venga prodotto al termine dell'algoritmo è la probabilità che **nessuno** dei k archi di C venga contratto durante le $n - 2$ iterazioni dell'algoritmo Randomized Min-Cut.

Per calcolare tale probabilità, conviene innanzitutto determinare **quanti archi contiene** il grafo G .

Algoritmo per il taglio minimo

Il grado minimo di G è k , perché se qualche vertice avesse grado $< k$, allora quegli archi costituirebbero un taglio più piccolo.

G ha almeno $\frac{nk}{2}$ archi, tra i quali si seleziona uniformemente a caso l'arco da contrarre.

$$P\{\text{nessun arco di } C \text{ è selezionato al 1° passo}\} \geq \frac{\frac{nk}{2} - k}{\frac{nk}{2}} = \frac{nk - 2k}{nk} = \frac{n - 2}{n}.$$

Supponiamo che, dopo il primo passo, non sia stato eliminato alcun arco di C . Rimane un grafo con $n - 1$ vertici. Questo grafo ha ancora un taglio minimo pari a C .

Perché?

Algoritmo per il taglio minimo

Abbiamo già osservato che qualsiasi cut-set del grafo contratto è anche un cut-set del grafo originale.

Dunque, se il grafo contratto avesse un taglio minimo di cardinalità $< |C|$, allora anche il grafo originale dovrebbe avere un cut-set di cardinalità $< |C|$, il che è una contraddizione.

Poiché il grafo contratto ha ancora un cut-set di cardinalità $|C|$, esso deve avere almeno $\frac{k(n-1)}{2}$ archi.

$$\begin{aligned} &P\{\text{nessun arco di } C \text{ è selezionato al 2° passo} \mid \text{nessun arco selezionato al 1°}\} \\ &\geq \frac{\frac{(n-1)k}{2} - k}{\frac{(n-1)k}{2}} = \frac{(n-1)k - 2k}{(n-1)k} = \frac{n-3}{n-1}. \end{aligned}$$

Algoritmo per il taglio minimo

Abbiamo già visto che

$$\mathbf{P}\{E_1\} \geq \frac{n-2}{n} \quad e \quad \mathbf{P}\{E_2 \mid E_1\} \geq \frac{n-3}{n-1}.$$

Con lo stesso argomento otteniamo

$$\mathbf{P}\{E_3 \mid E_1 \cap E_2\} \geq \frac{n-4}{n-2}.$$

Da cui segue:

$$\mathbf{P}\{\text{nessun arco di } C \text{ viene mai contratto}\} = \mathbf{P}\{E_1 \cap E_2 \cap \cdots \cap E_{n-2}\}$$

$$= \mathbf{P}\{E_1\} \mathbf{P}\{E_2 \mid E_1\} \mathbf{P}\{E_3 \mid E_1 \cap E_2\} \cdots \mathbf{P}\{E_{n-2} \mid E_1 \cap \cdots \cap E_{n-3}\}$$

$$= \frac{2}{n(n-1)}.$$

Algoritmo per il taglio minimo

Questo significa che, **in una singola esecuzione**, la probabilità di successo (ottenere un taglio minimo) è

$$\frac{2}{n(n-1)}.$$

In una singola esecuzione la probabilità che il taglio restituito non sia minimo è

$$\leq 1 - \frac{2}{n(n-1)}.$$

Come possiamo ottenere un taglio minimo con **alta probabilità**? Eseguiamo l'algoritmo $\Theta(n^2 \ln n)$ volte.

Algoritmo per il taglio minimo

Assertione 22.12 Se eseguiamo **Randomized Min-Cut** $\Theta(n^2 \ln n)$ volte e riportiamo il taglio di cardinalità minima ottenuto, allora con probabilità $> 1 - \frac{1}{n^2}$ il taglio restituito è effettivamente un **taglio minimo**.

Dim. La probabilità che l'output non sia un taglio minimo è al più

$$\begin{aligned} \left(1 - \frac{2}{n(n-1)}\right)^{n^2 \ln n} &\leq \left(1 - \frac{2}{n(n-1)}\right)^{n(n-1) \ln n} = \left[\left(1 - \frac{2}{n(n-1)}\right)^{n(n-1)}\right]^{\ln n} \\ &< e^{-2 \ln n} = \frac{1}{n^2}. \end{aligned}$$

Algoritmo per il taglio minimo

Conclusione. Il tempo totale di esecuzione dell'algoritmo **Randomized Min-Cut** comporta $\Theta(n^3 \ln n)$ contrazioni: un tempo di esecuzione molto buono.

L'algoritmo **Randomized Min-Cut** si distingue inoltre per la sua estrema **semplicità**.

Esiste, naturalmente, una (piccolissima) possibilità di errore; tuttavia, tale errore **non è fatale**, poiché l'algoritmo restituisce sempre un **insieme di taglio** valido.

Balls and bins

Balls and bins

Passiamo ora a un'applicazione molto diversa dei limiti di coda.

Abbiamo delle palline che vengono lanciate in modo uniforme e casuale in alcune urne.

Domanda tipica: In media quante palline dovrebbe contenere ciascuna urna?

Problemi di questo tipo compaiono spesso in informatica:

Load balancing, hashing, hashing crittografico,...

Balls and bins

Consideriamo il caso più semplice.

Abbiamo esattamente n palline, ciascuna lanciata uniformemente a caso in una delle n urne.

Domanda: In media, quante palline dovrebbe contenere ciascuna urna?

Risposta: Una pallina, in valore atteso.

Domanda: Qual è il numero massimo di palline che un'urna può contenere?

Risposta: n .

Alta probabilità (w.h.p.)

Definizione 19.5 — “With High Probability” (w.h.p.)

Il termine “**with high probability**” (*con alta probabilità*, abbrev. *w.h.p.*) indica in genere una probabilità dell'ordine di $1 - \frac{1}{n}$, dove n rappresenta la dimensione del problema.

Talvolta il termine è usato in senso un po' più ampio, per indicare una probabilità dell'ordine di $1 - \frac{1}{n^c}$, dove $c > 0$ è una costante.

Quando si formulano garanzie probabilistiche *w.h.p.*, è comune richiedere che n sia **sufficientemente grande**.

Carico massimo

Teorema 19.6 — Carico massimo

Per n palline lanciate in modo uniforme in n urne:

$$P\{\text{ogni urna ha } \leq k \text{ palline}\} \geq 1 - \frac{1}{n},$$

dove $k = \frac{3 \ln n}{\ln \ln n} - 1$ e n è sufficientemente grande.

Dim.

Consideriamo solo la j -esima urna. Sia B_j il numero di palline nell'urna j . Qual è la distribuzione di B_j ?

Carico massimo

Consideriamo solo la j -esima urna. Sia B_j il numero di palline nell'urna j . Qual è la distribuzione di B_j ?

Definiamo $X_i = \begin{cases} 1 & \text{Se la pallina } i\text{-esima va nell'urna } j \\ 0 & \text{Se la pallina } i\text{-esima non va nell'urna } j \end{cases}$

$$B_j = \sum_{i=1}^n X_i$$

Distribuzione del numero di palline in un'urna

$B_j \sim \text{Bin}(n, 1/n)$, quindi $E[B_j] = 1$

Carico massimo

Obiettivo (w.h.p.)

Mostrare che ogni urna ha $\leq k$ palline con alta probabilità.

Equivalentemente:

$$P\{\exists \text{ urna con } > k \text{ palline}\} < \frac{1}{n}.$$

Equivalentemente:

$$P\{B_1 > k \text{ oppure } B_2 > k \text{ oppure } \dots \text{ oppure } B_n > k\} < \frac{1}{n}.$$

Stimiamo $P\{B_j > k\}$ per un arbitrario j .

Carico massimo

Vogliamo dimostrare che

$$\mathbf{P}\{B_1 > k \text{ oppure } B_2 > k \text{ oppure } \dots \text{ oppure } B_n > k\} < \frac{1}{n}.$$

È sufficiente dimostrare che per un arbitrario j , $\mathbf{P}\{B_j > k\} < \frac{1}{n^2}$

Infatti, usando poi l'union bound:

$$\mathbf{P}\{B_1 > k \text{ oppure } B_2 > k \text{ oppure } \dots \text{ oppure } B_n > k\}$$

$$\leq \sum_{j=1}^n \mathbf{P}\{B_j > k\} < n \cdot \frac{1}{n^2} = \frac{1}{n}$$

Chernoff bound forte

Teorema 18.6 — Chernoff Bound (forma forte)

Per $X = \sum_i X_i$, dove $X_i \sim \text{Bernoulli}(p_i)$ e $\mu = \mathbf{E}[X]$:

$$\mathbf{P}\{X \geq (1 + \varepsilon)\mu\} < \left(\frac{e^\varepsilon}{(1 + \varepsilon)^{1+\varepsilon}} \right)^\mu.$$

- È più generale del **Teorema 18.4** perché le X_i possono avere parametri diversi: p_i .
- È più forte perché il limite $\left(\frac{e^\varepsilon}{(1+\varepsilon)^{1+\varepsilon}} \right)^\mu$ risulta spesso migliore di quello del **Teorema 18.4**.

Chernoff bound forte

Teorema 18.6 — Chernoff Bound (forma forte)

Per $X = \sum_i X_i$, dove $X_i \sim \text{Bernoulli}(p_i)$ e $\mu = E[X]$:

$$P\{X \geq (1 + \varepsilon)\mu\} < \left(\frac{e^\varepsilon}{(1 + \varepsilon)^{1+\varepsilon}} \right)^\mu.$$

Nel nostro caso tutte le B_j hanno **lo stesso parametro** $p_j = 1/n$;
quindi $B_j \sim \text{Bin}(n, 1/n)$, cioè **tutte le B_j sono i.i.d.** con media $\mu = 1$;
dunque **non serve la generalità** del Teorema 18.6 (che ammette p_j diversi).

Lo usiamo solo perché fornisce **una forma più stretta del limite superiore**.

Carico massimo

Teorema 18.6 — Chernoff Bound (forma forte)

Per $X = \sum_i X_i$, dove $X_i \sim \text{Bernoulli}(p_i)$ e $\mu = E[X]$:

$$\mathbf{P}\{X \geq (1 + \varepsilon)\mu\} < \left(\frac{e^\varepsilon}{(1 + \varepsilon)^{1+\varepsilon}} \right)^\mu.$$

Nel nostro caso abbiamo già visto che $\mu = 1$:

$$\mathbf{P}\{B_j > k\} = \mathbf{P}\{B_j \geq 1 + k\} = \mathbf{P}\{B_j \geq (1 + \varepsilon)\mu\} \quad \text{con } \varepsilon = k \text{ e } \mu = 1.$$

$$< \frac{e^k}{(1 + k)^{1+k}}.$$

Carico massimo

$$\mathbf{P}\{B_j > k\} = \mathbf{P}\{B_j \geq 1 + k\} = \mathbf{P}\{B_j \geq (1 + \epsilon)\mu\} \quad \text{con } \epsilon = k \text{ e } \mu = 1.$$

$$< \frac{e^k}{(1 + k)^{1+k}}.$$

Dunque, per dimostrare che

$$\mathbf{P}\{B_j \geq 1 + k\} < \frac{1}{n^2},$$

è sufficiente provare che

$$\frac{e^k}{(1 + k)^{1+k}} \leq \frac{1}{n^2}.$$

Carico massimo

Prendendo i logaritmi di entrambi i membri:

$$\frac{e^k}{(1+k)^{1+k}} \leq \frac{1}{n^2} \quad \Leftrightarrow \quad k - (1+k) \ln(1+k) \leq -2 \ln n.$$

Sostituendo $k = \frac{3 \ln n}{\ln \ln n} - 1$, otteniamo:

$$\frac{3 \ln n}{\ln \ln n} - 1 - \frac{3 \ln n}{\ln \ln n} \cdot \ln \left(\frac{3 \ln n}{\ln \ln n} \right) \leq -2 \ln n.$$

Carico massimo

Espandendo $\ln\left(\frac{3 \ln n}{\ln \ln n}\right)$:

$$\frac{3 \ln n}{\ln \ln n} - 1 - \frac{3 \ln n}{\ln \ln n} \cdot (\ln 3 + \ln \ln n - \ln \ln \ln n) \leq -2 \ln n .$$

Dividendo entrambi i membri per $\ln n$:

Che equivale a:

$$\frac{3}{\ln \ln n} - \frac{1}{\ln n} - \frac{3 \ln 3}{\ln \ln n} - 3 + \frac{3 \ln \ln \ln n}{\ln \ln n} \leq -2.$$

Carico massimo

$$\frac{3}{\ln \ln n} - \frac{1}{\ln n} - \frac{3 \ln 3}{\ln \ln n} - 3 + \frac{3 \ln \ln \ln n}{\ln \ln n} \leq -2.$$

Per n sufficientemente grande, tutti i termini frazionali sono $o(1)$, quindi:

$$o(1) + o(1) + o(1) - 3 + o(1) \leq -2.$$

Quindi, **per n sufficientemente grande**, la disuguaglianza $\frac{e^k}{(1+k)^{1+k}} \leq \frac{1}{n^2}$ è soddisfatta.

Carico massimo

Quindi, **per n sufficientemente grande**, la disuguaglianza $\frac{e^k}{(1+k)^{1+k}} \leq \frac{1}{n^2}$ è soddisfatta. Cioè $\mathbf{P}\{B_j \geq 1 + k\} < \frac{1}{n^2}$, per n sufficientemente grande.

Applichiamo l'union bound:

$$\mathbf{P}\{B_1 > k \text{ oppure } B_2 > k \text{ oppure } \dots \text{ oppure } B_n > k\}$$

$$\leq \sum_{j=1}^n \mathbf{P}\{B_j > k\} < n \cdot \frac{1}{n^2} = \frac{1}{n}$$

Hashing

Esempio

Sistema che memorizza info accademiche per studente (nome, corso di laurea, voti, ...).
Ogni studente è identificato da una **chiave** → il suo numero di matricola.
Inserendo la *chiave* ⇒ si ottengono le *info* corrispondenti.

Operazioni richieste:

- **Ricerca;**
- **Inserimento** (nuovi studenti);
- **Cancellazione** (laureati).

Hashing

Supponiamo che ci siano $m = 20000$ studenti. Come memorizzare le info per ciascuno degli studenti?

Una soluzione è usare una **lista** o un **array non ordinato**. In tal caso l'inserimento è veloce, ma **ricerca** e **cancellazione** devono scansionare linearmente l'intera lista, richiedendo tempo $O(m)$.



Inserimento: $O(1)$

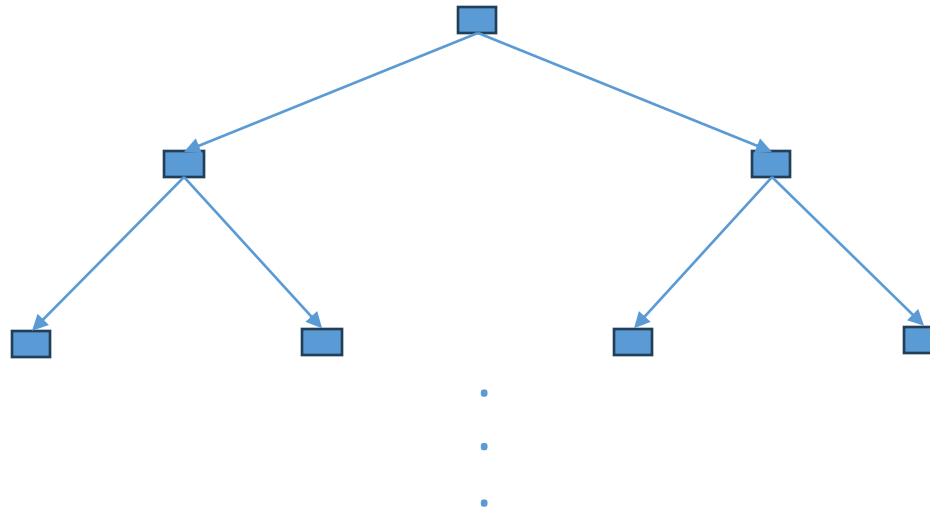
Ricerca: $O(m)$

Cancellazione: $O(m)$

Hashing

Una soluzione migliore è usare una struttura **ordinata**, ad esempio un **albero binario di ricerca** che ordina le informazioni degli studenti per **numero di matricola**.

Così **ricerca**, **inserimento** e **cancellazione** richiedono tutti tempo $O(\log m)$ in media.



Inserimento: $O(\log m)$

Ricerca: $O(\log m)$

Cancellazione: $O(\log m)$

Possiamo fare ancora meglio?

Hashing

Se lo spazio non fosse un problema, si potrebbe usare un **array enorme** A in cui la **matricola** è l'indice dell'array.

Per esempio, se la matricola di Marco è 123456789, le sue informazioni sono memorizzate in $A[123456789]$. In questo modo, i tempi di **search**, **insert** e **delete** sono tutti $O(1)$.

Tuttavia, se ci sono 10^9 possibili valori di matricola, la dimensione di A deve essere 10^9 .

È un **enorme spreco di spazio** per memorizzare i dati di **solì 20.000 studenti**.

Verso $O(1)$ medio con $O(m)$ spazio

Supponiamo di rinunciare alle garanzie di **caso peggiore**. Esiste una soluzione con **tempo medio** $O(1)$ per *search*, *insert* e *delete* che usi solo spazio $O(m)$?

Idea. Dividiamo gli studenti in $n = 10$ **bucket** in base all'ultima cifra della loro **matricola**. Tutti gli studenti con matricola che termina in 0 vanno nel bucket 0, quelli che terminano in 1 nel bucket 1, e così via.

912124120	102124121	022123122	103134123	112984124	233124105	022322106	123134127	033322108	134134129
912231140	231231141	472234142	231453113	921331144	743231105	472134126	231113117	471124128	009113119
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
0	1	2	3	4	5	6	7	8	9

Verso $O(1)$ medio con $O(m)$ spazio

Se vogliamo cercare Marco, sappiamo che la sua matricola finisce per 9, quindi appartiene al **bucket 9** e dobbiamo guardare solo lì.

912124120	102124121	022123122	103134123	112984124	233124105	022322106	123134127	033322108	134134129
912231140	231231141	472234142	231453113	921331144	743231105	472134126	231113117	471124128	009113119
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
0	1	2	3	4	5	6	7	8	9

Se i bucket sono bilanciati: ~2000 studenti/bucket → **speedup ×10** rispetto a una sola lista.

Domanda: possiamo generalizzare?

Verso $O(1)$ medio con $O(m)$ spazio

Aumentiamo il numero di **bucket** per migliorare ulteriormente il tempo di ricerca.

Per esempio, possiamo usare le **ultime quattro cifre** della matricola. Avremo così **10.000 bucket**, con suffissi da 0000 a 9999.

Quindi, per cercare Marco, basta guardare nel **bucket 6789** che, assumendo bucket di dimensione approssimativamente uguale, contiene in valore atteso di $\frac{20000}{10000} = 2$ studenti.

Soluzione: usare $n = O(m)$ bucket, il che consente di ottenere **ricerca $O(1)$ in media** con **spazio $O(m)$** !

Bucket hashing

Definizione — Funzione hash (mappa le **chiavi** nei **bucket**).

$$h : U \rightarrow B$$

- U : Universo di tutte le chiavi (insieme **molto grande**).
- B : Insieme dei **bucket** (es. $0, \dots, n - 1$).
- $K \subseteq U$: Chiavi effettive ($|K| = m$, $|K| \ll |U|$).
- B_i : Numero di chiavi che cadono nel bucket i .

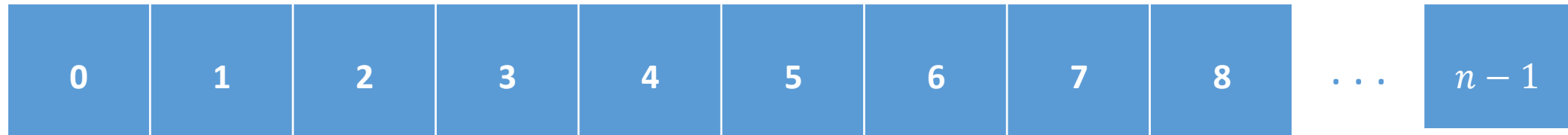
La **struttura dati** che mappa le m chiavi in n bucket mediante una funzione di hash è detta **tabella hash** (*hash table*).

Bucket hashing

Universo U (tutte le chiavi)



funzione $h(k)$



Load factor

Quando abbiamo modificato il numero di bucket, stavamo in realtà **bilanciando lo spazio con il tempo di ricerca**.

Definizione 20.2 — Fattore di carico

Una tabella hash che memorizza m chiavi in n bucket si dice avere **fattore di carico**

$$\alpha = \frac{\text{Numero di chiavi}}{\text{Numero di bucket}} = \frac{m}{n}.$$

È tipico mantenere un **fattore di carico** che sia una **costante di poco superiore a 1**.

Funzione hash

Proprietà desiderabili di una funzione hash

- **Efficiente** da calcolare.
- **Bilanciata** → distribuisce le chiavi *uniformemente* tra i bucket.

Esempio: $h(k) = k \bmod n$

- Buona se i valori numerici delle chiavi sono **uniformi rispetto ai resti mod n** .
- Non buona se essi hanno una **struttura regolare** (es. molti numeri che terminano con le stesse cifre).

La costruzione di funzioni hash efficienti e bilanciate **dipende in genere dal contesto**.

Assunzione SUHA

Ai fini dell'analisi, si assume semplicemente che la nostra funzione hash sia **efficiente** e rispetti una proprietà di bilanciamento nota come **assunzione di hashing uniforme** (*Simple Uniform Hashing Assumption* - SUHA).

Le chiavi si comportano come **palline lanciate casualmente e indipendentemente** in n urne.

Assunzione SUHA

Definizione 20.3 — Assunzione di hashing uniforme (SUHA)

Una funzione di hash h soddisfa la **Simple Uniform Hashing Assumption (SUHA)** se ogni chiave k ha probabilità $\frac{1}{n}$ di essere assegnata a qualunque bucket $b \in B$, dove $|B| = n$.

Inoltre, i valori di hash di chiavi diverse sono **indipendenti**, cioè per ogni insieme di chiavi $k_1, k_2, \dots, k_i \in K$, con $k_1 \neq k_2 \neq \dots \neq k_i$, e per ogni insieme di bucket

Funzione hash

Dato che $h(k)$, il valore di hash della chiave k , è **deterministico**, come possiamo affermare che $h(k) = b$ con probabilità $1/n$?

Si usa una **famiglia universale di funzioni hash** $\{h_1, h_2, \dots, h_n\}$.

Si sceglie **una funzione uniformemente a caso** e la si usa per tutte le chiavi.

Una volta scelta una funzione h_i , **tutte le chiavi della tabella** vengono mappate usando quella stessa funzione.

In questo modo la funzione hash rimane **deterministica**, ma possiede **proprietà statistiche casuali** adatte all'analisi.

Funzione hash

Sia n il numero di bucket e m il numero di chiavi, con $\alpha = \frac{m}{n}$.

Definiamo $I_k = \begin{cases} 1 & \text{se } h(k) = i \text{ (la chiave } k \text{ viene mappata nel bucket } i). \\ 0 & \text{altrimenti.} \end{cases}$

$B_i = \sum_{k=1}^m I_k$ numero di chiavi nell' i -esimo bucket.

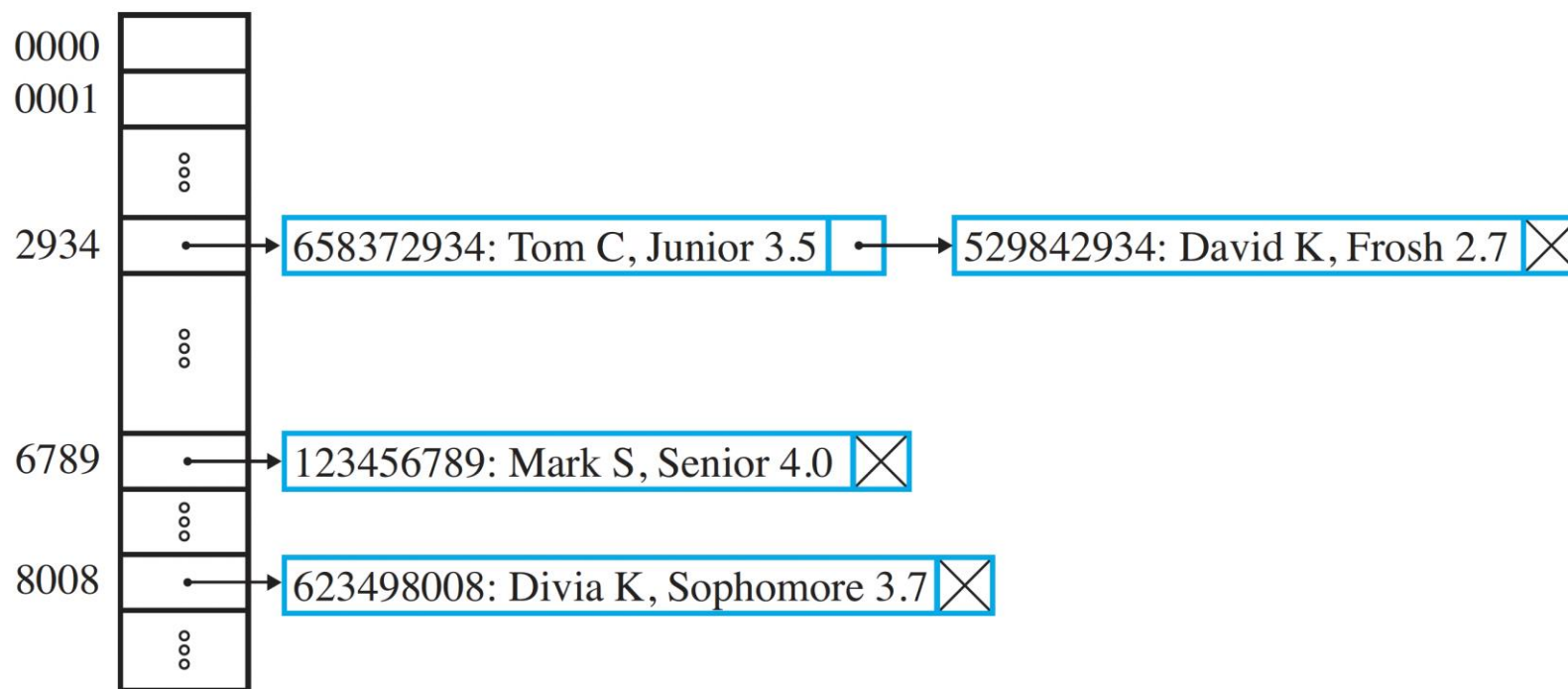
Per la **linearità del valore atteso**:

$$E[B_i] = \sum_{k=1}^m E[I_k] = \sum_{k=1}^m \frac{1}{n} = \frac{m}{n} = \alpha.$$

Quindi tutti i bucket hanno, in media, la stessa dimensione α .

Hashing con Separate Chaining

Nel **bucket hashing con concatenamento separato**, la **tabella hash** è un **array di bucket**, dove ogni bucket i mantiene una **lista di chiavi** che contiene tutte le chiavi k tali che $h(k) = i$.



Hashing con Separate Chaining

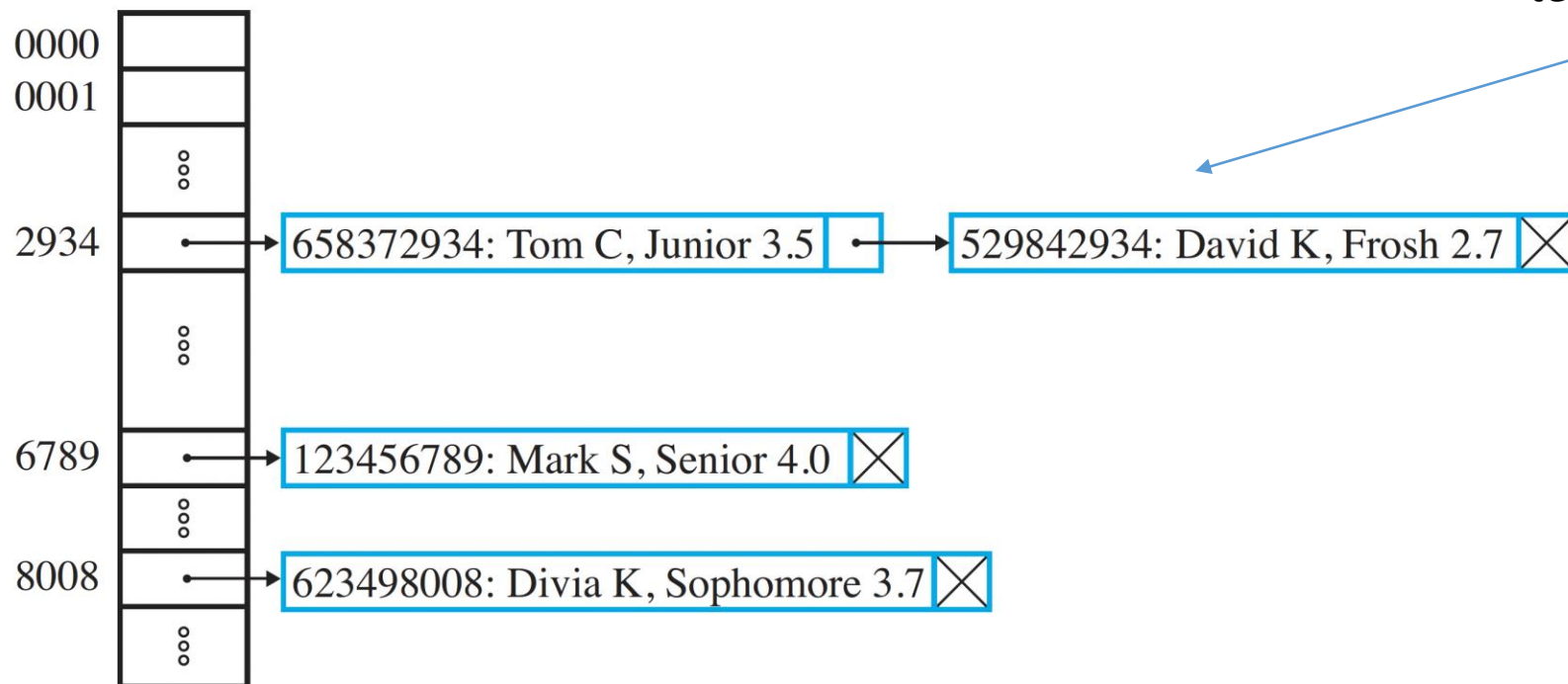
Operazioni:

Search → percorri la lista

Insert → cerca, poi aggiungi se non presente

Delete → cerca, poi rimuovi

Complessità dominata dal
tempo di **ricerca** nel bucket



Hashing con Separate Chaining

Abbiamo già visto che, sotto SUHA e assumendo un fattore di carico α , ogni bucket ha α chiavi in valore atteso.

Quindi, il **tempo atteso di ricerca** nel bucket hashing con concatenamento separato è $O(\alpha)$.

Questo è ottimo, perché in genere immaginiamo α come una piccola costante. Tuttavia, **un singolo bucket** potrebbe contenere **molto più** di α chiavi.

Domanda: Qual è la **distribuzione** di B_i , il numero di chiavi nell' i -esimo bucket?

Ricorda che stiamo distribuendo m chiavi in n bucket, uniformemente a caso.

Hashing con Separate Chaining

$$B_i \sim \text{Binomiale}\left(m, \frac{1}{n}\right).$$

Supponiamo che m e n siano entrambi grandi, mentre α resta costante. Cosa possiamo dire di $\mathbf{Var}(B_i)$?

Risposta:

$$\mathbf{Var}(B_i) = m \cdot \frac{1}{n} \cdot \left(1 - \frac{1}{n}\right) = \alpha \left(1 - \frac{1}{n}\right) \rightarrow \alpha.$$

Per n e m sufficientemente grandi

Hashing con Separate Chaining

$$B_i \sim \text{Binomiale}\left(m, \frac{1}{n}\right).$$
$$\text{Var}(B_i) = m \cdot \frac{1}{n} \cdot \left(1 - \frac{1}{n}\right) = \alpha \left(1 - \frac{1}{n}\right) \rightarrow \alpha \cdot \mathbf{E}[B_i] = \alpha.$$

Quando m e n sono grandi, il **CLT** ci dice che la distribuzione di B_i **si avvicina** a una Normale.

Domanda: Dunque, quando m e n sono grandi, quanto vale *approssimativamente* $P\{B_i > \alpha + 2\sqrt{\alpha}\}$?

Hashing con Separate Chaining

$P\{B_i > \alpha + 2\sqrt{\alpha}\}$ è la probabilità che B_i superi la sua media (che è α) di più **2 deviazioni standard** ($2\sqrt{\alpha}$). Poiché la distribuzione di B_i tende a una Normale, tale probabilità è **circa il 2%**:

$$P\{B_i > \alpha + 2\sqrt{\alpha}\} \approx 1 - \Phi(2) \approx 0.0228.$$

Dunque, il numero di chiavi in un singolo bucket tende a essere piccolo: la media è α e, per m e n grandi, la distribuzione si avvicina alla Normale(α, α).

Ma che dire del bucket con il più alto carico? Quante chiavi contiene?

Hashing con Separate Chaining

Quando $\alpha = 1$, abbiamo $m = n$. Abbiamo mostrato che, se si lanciano n palline in n urne in modo uniforme e casuale, allora **con alta probabilità** (w.h.p.) l'urna più piena contiene $O\left(\frac{\ln n}{\ln \ln n}\right)$ palline.

Questo fornisce un **limite w.h.p. sul costo di ricerca** nel caso $\alpha = 1$.

Possiamo immaginare di ottenere limiti simili (w.h.p.) anche per i casi $\alpha = 2$ o $\alpha = 3$.

Ma cosa accade se α è grande, ad esempio $\alpha = \ln n$?

Si potrebbe pensare che il bucket più carico possa contenere molte più chiavi.

Hashing con Separate Chaining

Teorema 20.4

Sotto l'assunzione SUHA, per il **bucket hashing con concatenamento separato**, supponendo $m \geq 2n \ln n$ chiavi e n bucket, allora con probabilità almeno $1 - \frac{1}{n}$ il bucket più grande ha dimensione minore di $e\alpha$, dove $\alpha = \frac{m}{n}$.

Morale: anche con fattori di carico grandi, il bucket più pieno resta di ordine $O(\alpha)$.

Strategia della dimostrazione.

Per ogni i dimostriamo prima che $P\{B_i \geq e\alpha\} \leq \frac{1}{n^2}$

Poi usiamo l'**union bound**: $P\{\exists i: B_i \geq e\alpha\} \leq n \cdot \frac{1}{n^2} = \frac{1}{n}$

$$\Rightarrow P\{\max \text{ bucket} < e\alpha\} \geq 1 - \frac{1}{n}.$$

Hashing con Separate Chaining

Teorema 20.4

Sotto l'assunzione SUHA, per il **bucket hashing con concatenamento separato**, supponendo $m \geq 2n \ln n$ chiavi e n bucket, allora con probabilità almeno $1 - \frac{1}{n}$ il bucket più grande ha dimensione minore di $e\alpha$, dove $\alpha = \frac{m}{n}$.

Dim. È sufficiente, allora, dimostrare che per ogni i : $\mathbf{P}\{B_i \geq e\alpha\} \leq \frac{1}{n^2}$.

Cominciamo osservando che, poiché $m \geq 2n \ln n$, abbiamo

$$\alpha = \frac{m}{n} \geq 2 \ln n.$$

Applichiamo quindi il **Chernoff Bound** forte, scegliendo $1 + \varepsilon = e$ (quindi $\varepsilon = e - 1 > 0$), e $\mu = \alpha \geq 2 \ln n$.

Hashing con Separate Chaining

$$\mathbf{P}\{B_i \geq e\alpha\} = \mathbf{P}\{B_i \geq (1 + \varepsilon)\mu\} < \left(\frac{e^\varepsilon}{(1 + \varepsilon)^{1+\varepsilon}} \right)^\mu.$$

Sostituendo $1 + \varepsilon = e$ e $\mu = \alpha$:

$$\mathbf{P}\{B_i \geq e\alpha\} < \left(\frac{e^{e-1}}{e^e} \right)^\alpha = (e^{-1})^\alpha = e^{-\alpha}.$$

Poiché $\alpha \geq 2 \ln n$:

$$e^{-\alpha} \leq e^{-2 \ln n} = \frac{1}{n^2}.$$

Dunque,

$$\mathbf{P}\{B_i \geq e\alpha\} \leq \frac{1}{n^2}.$$